product brief + system design, sketched

Course-to-Book Engine

Turn a playlist of recorded lectures into a real book: organised by idea, every fact traceable to the moment it was said, and rebuildable exactly.

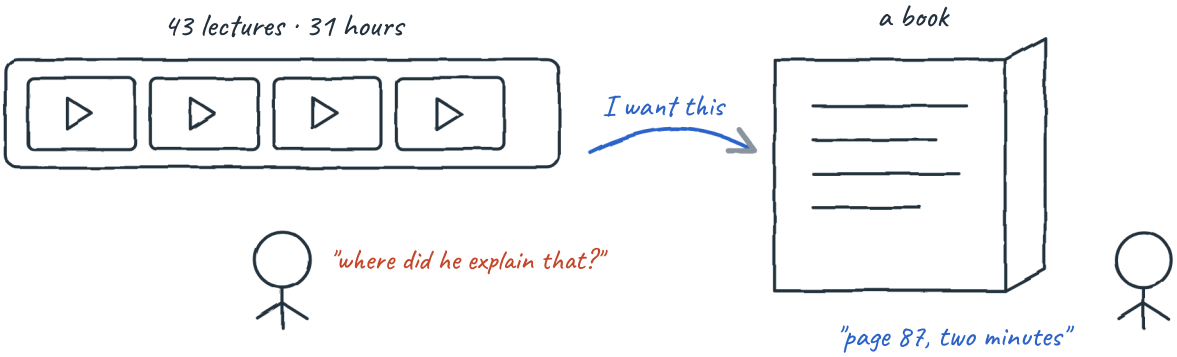
In thirty seconds

- Built from the 22 public lectures of a 43-lecture course: **a 164-page book, 6 chapters, 52,000 words**, plus a web edition where every paragraph links to the second of video it came from.
- **An independent AI evaluator backs 96.5%** of a fresh random sample of its factual sentences (target 95%), finds 1.5 misleading errors per 100 (target at most 2), and misses 4.3% of core ideas (target at most 10%).
- Every flagged sentence was decided by a human reviewer. Running the course's own code caught two numbers the lecture got wrong.
- Nothing course-specific is built in: a new course needs a new settings card, not new code.
- Anyone can rebuild the identical book from the repository without an API key.

Source code on GitHub

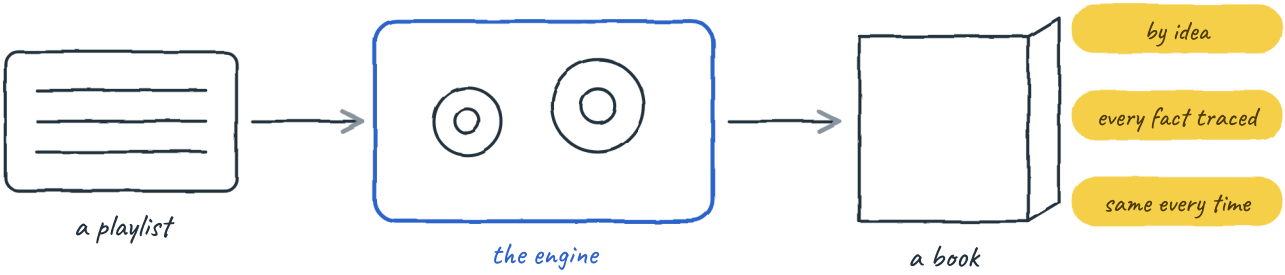
Part 1 · The product

1 The problem



Courses are good for learning in order and bad for looking things up. A book fixes that, but writing one takes weeks, and a book written quickly by an AI model can't be trusted: nothing shows which sentences the lecturer actually said.

2 What I am building



The engine reads a lecture playlist and writes a book with chapters, an introduction, short summaries and diagrams. Three promises hold for every book it makes: it is **organised by idea**, not by video; **every fact can be traced** to the second of video it came from; and running it again gives **the same book**.

3 The thinking behind it: six questions

Before any design, the problem was looked at through six questions. Each one produced a decision, and each decision now lives in the system as a small markdown file that is regenerated with every book.

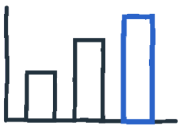


who / what · portrait

Who is it for, and what must it teach?

Work out the reader from the course itself, never a stock persona. Unknowns stay unknown; an explicit brief wins over a guess.

→ *reader.md*



how much · chart

How much of the course must reach the book?

Keep a ledger of every idea taught: placed in the book or left out with a reason. Measure trust with planted false facts.

→ *coverage.md*



where · map

Where does each idea belong?

Group by topic while keeping the teacher's order. Every lecture maps to chapters and sections, traceable to the second.

→ *concept_map.md*



when · timeline

When does the reader meet each idea, and when is the book ready?

Two clocks: the reading order (with the source minutes behind it), and readiness gates for the build. Only what changed is redone.

→ *learning_path.md*



how · flowchart

How is it made?

As a row of small, checkable steps on one memory shelf, so every step can be inspected, resumed and rebuilt.

→ *pipeline.md*



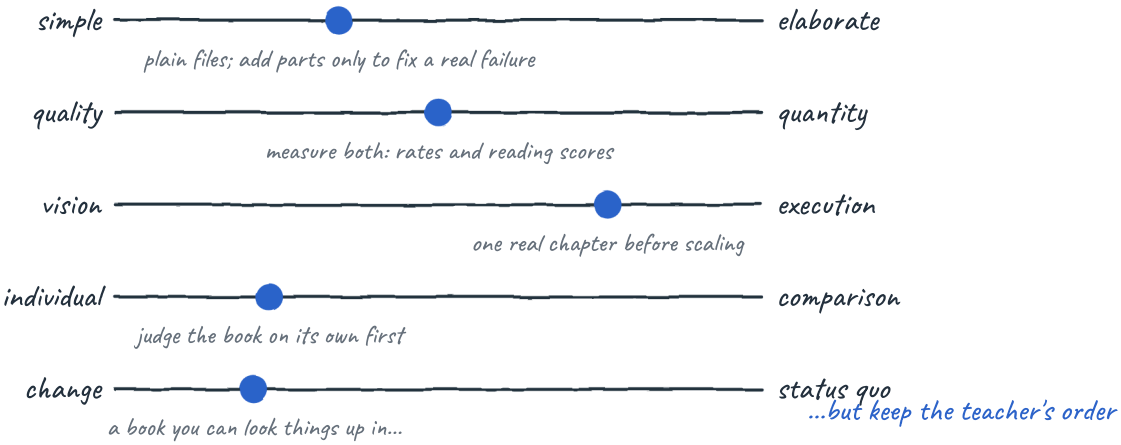
why · plot

Why trust it, and why this design?

Release only when measured rates meet their targets and the independent evaluator agent agrees; record every trade-off.

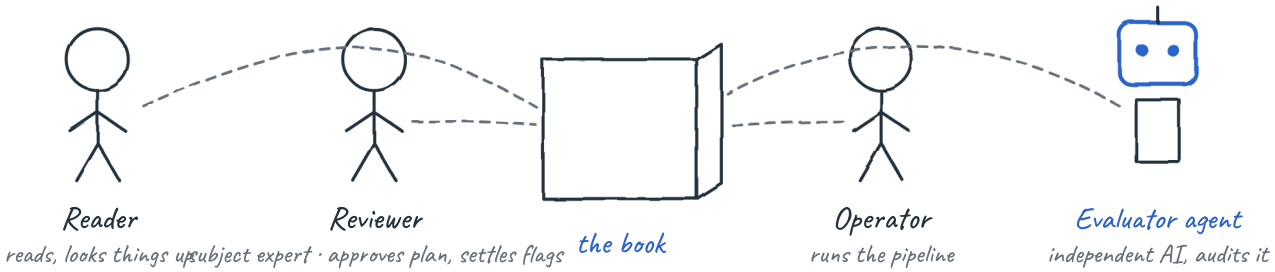
→ *rationale.md*

4 Five trade-offs I chose



Each slider is a choice made on purpose. The design stays as simple as the checks allow, measures both numbers and reading quality, proves one real chapter before scaling, judges the book on its own before comparing it with alternatives, and changes how the course is used while keeping how it was taught.

5 Who it is for



6 What it must do, and what it won't

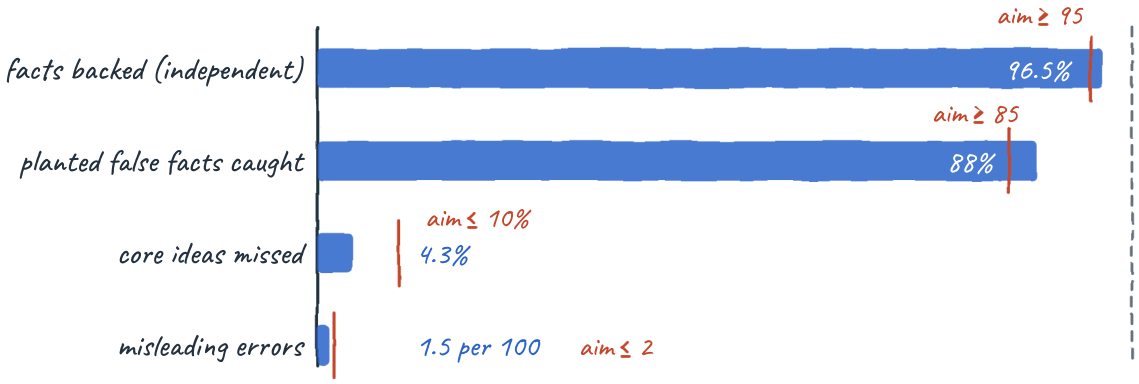
Must

- ✓ Read like a book: chapters by topic, an introduction, a quick summary before each chapter, takeaways at the end.
- ✓ Cover the whole course: every idea is placed somewhere or left out with a written reason.
- ✓ Back every fact with the lecture moment it came from, and check it.
- ✓ Say plainly how reliable the checking is, with numbers.
- ✓ Rebuild the same book for anyone who reruns it.
- ✓ Work on any lecture playlist by changing settings, not code.
- ✓ Ask the reviewer only about the few sentences the check can't settle.

Won't (yet)

- ✗ Correct the lecturer on its own. Where the course's executed code disagrees with what was said, the reviewer decides.
- ✗ Read whole slide decks. Only selected board frames and the course's own code are used.
- ✗ Pull facts from the open web.
- ✗ Build books from a cloud server. YouTube blocks caption requests from data centres, so builds run on a normal connection.

7 How I know it works



Red marks are the targets; blue bars are the final results on the full book. "Facts backed" is judged by the independent evaluator on a fresh random sample of 200 sentences, each judged three times. "Planted false facts" means I deliberately slip wrong sentences in and count how many the checker catches. That is how I know the checking actually works.

7½ What came out

Building LLMs from Scratch, Public Lectures Edition. A 43-lecture course (31 hours); 22 lectures have public captions, and the 21 members-only lectures are named in the book with their reason.

164
pages · 6 chapters · 78 sections

52,243
words, 16 diagrams

2,145
factual sentences, each citing caption lines

96.5%
backed, judged by the independent evaluator

106 / 120
planted false facts caught

37 / 46
code listings executed; 69 stated numbers confirmed

372
flagged items, every one decided by the reviewer

~$50
in AI calls for the whole book, including all checks

7¾ What went wrong, and what I did

Each of these was found by the system's own checks or by reading the output, not after the fact.

Half the course is behind a paywall

Every odd-numbered lecture is members-only, so its captions can't be read.

Declared, not hidden

The book is a "Public Lectures Edition, 22 of 43". Each missing lecture is listed with the reason YouTube gives, and full coverage needs one option and a membership, no code.

The lecture said 610 and 107

Running the course's code gives 6100 and 1107. The captions faithfully record a misread.

Code as evidence, reviewer decides

The disagreement went to the reviewer with the executed values; the text was corrected and the code run is recorded as the reason.

The evaluator disagreed with itself

Judging the same sentences twice gave different verdicts on about 10% of them, so a single 60-sentence sample couldn't tell 93% from 95%.

Measure better, never lower the bar

Each sampled sentence is now judged three times and the majority counts, over 200 sentences. The 95% target never moved, and the report states the method.

The coverage check punished a good book

It only counted an idea as covered if it was taught right where that lecture said it. A book that teaches each idea once looked like it was missing repeats.

Search the whole book, report both

Missing ideas are now searched across the book before they count. Both numbers are printed: 4.3% missed from the book, 12.8% near the clip.

Rewrites copied caption mistakes

An automatic rewording wrote "a Jou activation" because the captions misheard GELU.

Rules, then a second check

Rewordings must use the correct technical term and textbook voice, and every one is re-checked by the strict evaluator before the reviewer sees it.

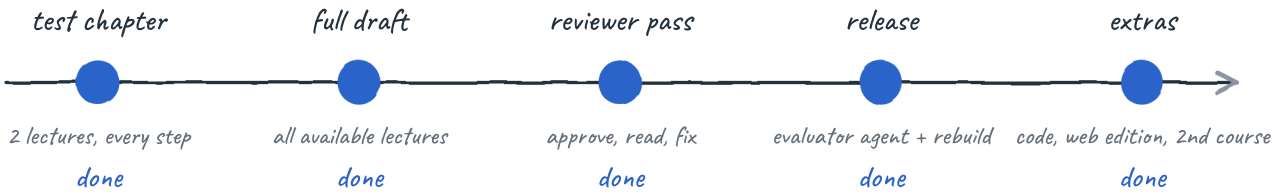
YouTube rate-limited my connection

After many requests, captions stopped coming back, even for lectures that had worked before.

Named reasons, resumable capture

Each missing lecture records why ("blocked by YouTube", "members-only"). Capture skips what it already has, so switching network and rerunning finished the job.

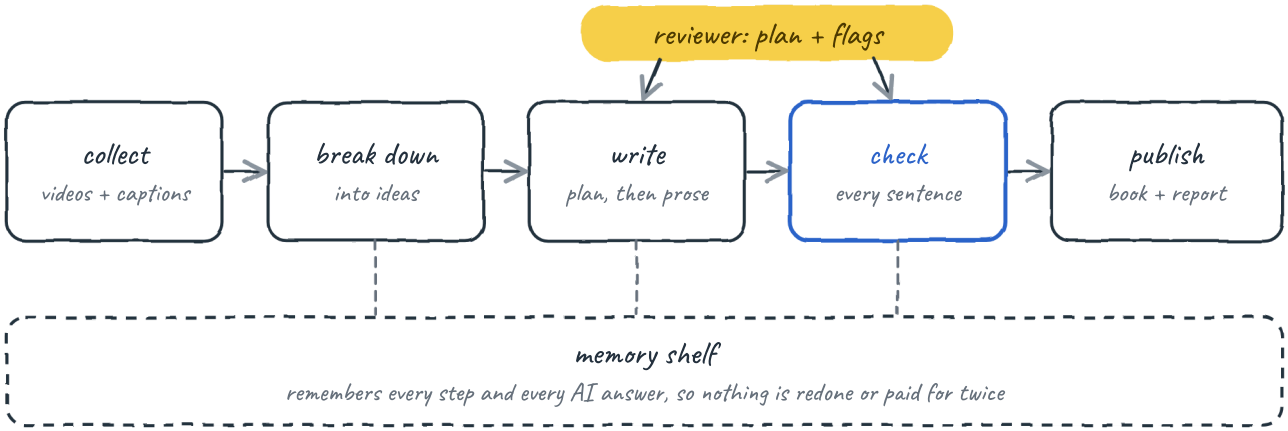
8 The plan



One real chapter first, then the whole book, then a careful human read, and only then the extras. Until every release condition was met, the book printed a "draft" page listing what was still missing; it now prints as a reviewed release.

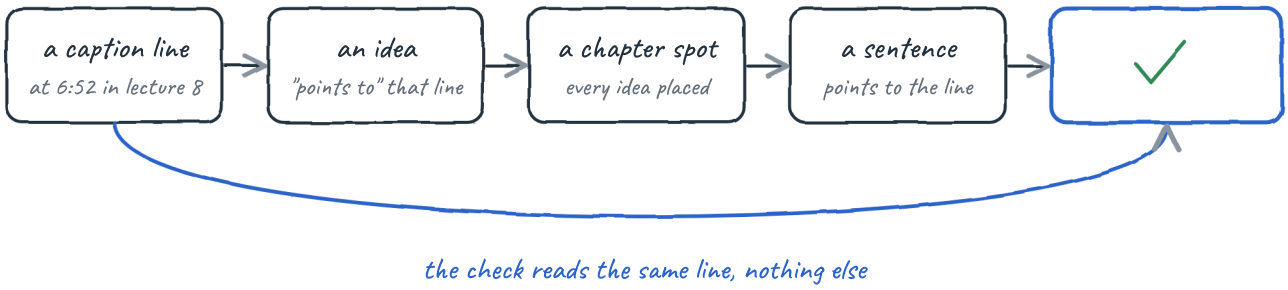
Part 2 · How it works

9 The big picture



Five steps in a row, the reviewer stepping in at two points, and a memory underneath everything. The rest of this page zooms into these boxes.

10 The thread that ties it together



Every caption line gets a label made from its video and the second it starts. Ideas, chapter plans and sentences all point back to those labels. That one thread is what makes every fact traceable: pick any sentence and you can play the exact moment it came from.

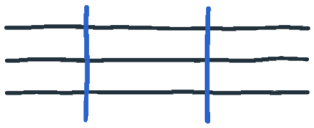
11 Each part, in a sentence or two



1

Collect

Save the playlist in order and keep a sealed copy of each lecture's captions. Lectures that can't be fetched are listed as gaps, not guessed.



2

Label and slice

Give every caption line its time label, then cut the lecture into short windows and quarter-hour chunks the AI can read whole.



3

Find the ideas

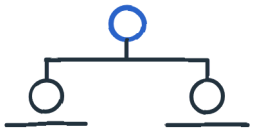
Pull out each thing taught (a definition, a step, an example) as a short sentence, with how important it is and where it was said.



3½

Profile the reader

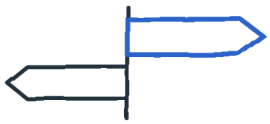
Work out who the book is for from the course itself: the goal, what the reader already knows, what is unknown. Every claim points to evidence; the result is **reader.md**.



4

Plan the book

Group lectures into chapters by topic, then sort each chapter's ideas into sections. Every idea is placed or left out with a reason. The reviewer approves the plan.



5

Set the scene

Write the introduction, a short opener and an "at a glance" box for each chapter, so the reader knows where they're going before the detail.



6

Write

Turn each section's ideas into connected book prose in the book's own voice. Every fact points to the caption lines it came from.



7

Check

A second, separate AI reads each sentence next to only the lines it cites and says whether they back it up. Weak paragraphs get one rewrite; leftovers go to the reviewer.



8

Draw diagrams

Sketch the processes and structures as diagrams. Each arrow must match a checked sentence, or the diagram is fixed once or left out.



9

Test the checker

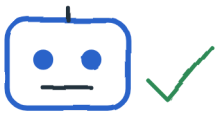
Slip in deliberately false sentences (a wrong number, a swapped term) and count how many the checker catches. The misses are printed in the book.



10

Reviewer decides

The reviewer sees each flagged sentence with the exact lecture lines and a link to the moment. Keep it, fix it (and it's re-checked), or cut it.



10½

Evaluator agent

A third, independent AI that never sees the pipeline's own verdicts audits the finished book: spot-checks facts, hunts for missed ideas in random clips, and scores each chapter.



11

Publish

Lay out the book as a PDF with chapters, boxes and figures, plus a short report of the measured numbers and a list of anything left out.



12

Code from the reference

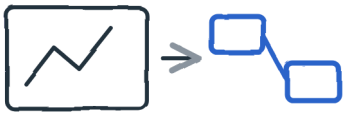
For sections that teach code, real listings are pulled word-for-word from the course's companion repository, pinned to one version, with file and licence shown.



13

Cross-check

Technical sentences are compared with the reference. Real disagreements go to the reviewer: keep the lecture, show both with a note, or edit.



14

Redraw the board

Where the lecturer points at the screen, one frame is grabbed, redrawn as a clean diagram or equation, then checked against the image and against what was said.

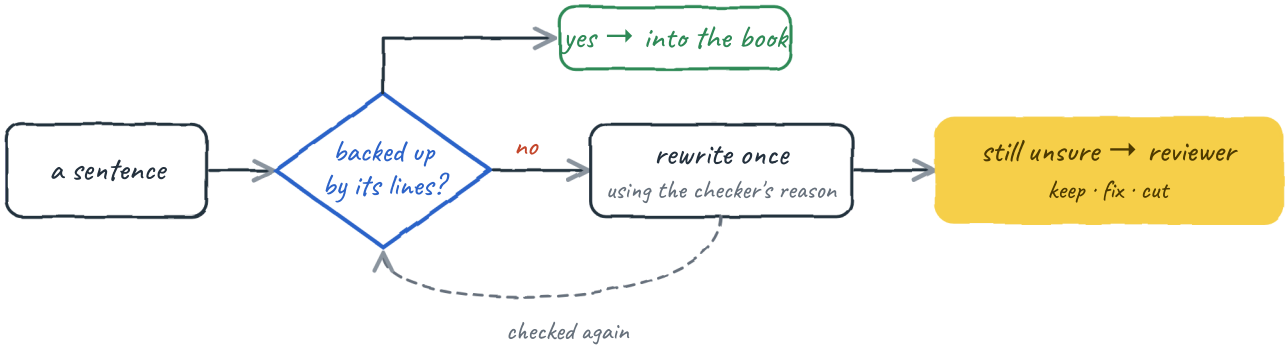


★

Memory shelf

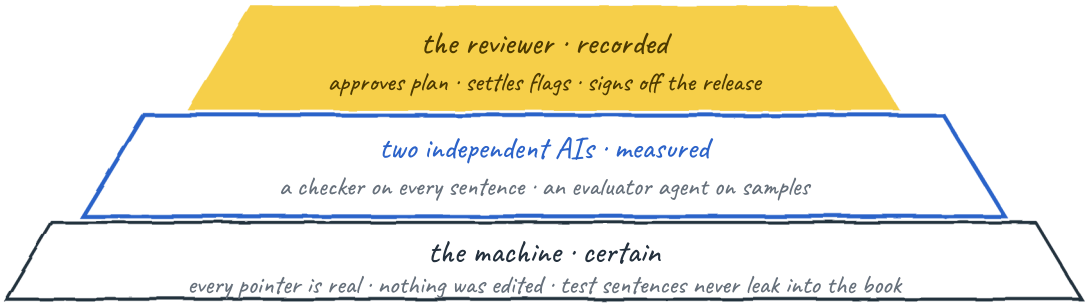
Every step's result and every AI answer is stored with a fingerprint of what went in. Same inputs, same answer, no new cost.

12 How one sentence gets checked



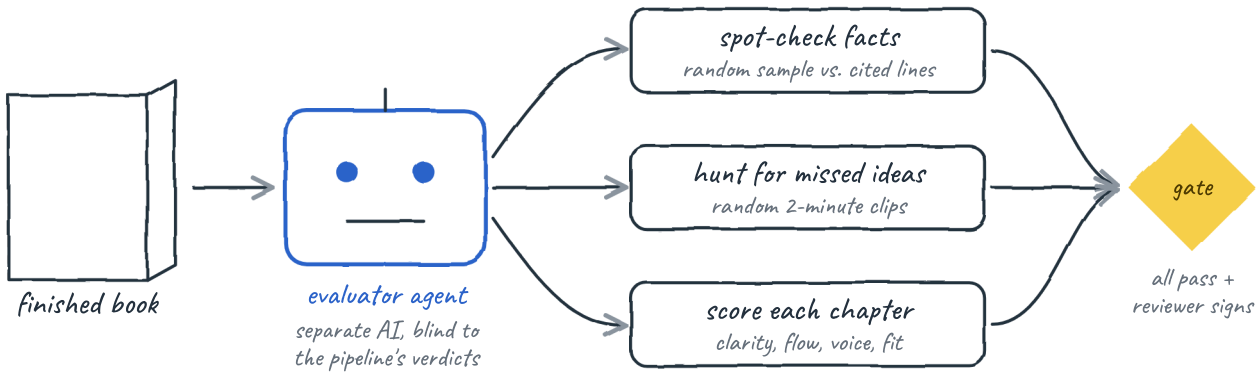
Nothing gets in on the writer's word alone. A sentence is either backed by the lines it points to, fixed until it is, or decided by the reviewer, who can see the evidence. If a later run changes that sentence, the decision is reopened.

13 Who decides what



Each kind of check sits where it's reliable. The machine does what it can do with certainty. The AI checker does the reading, and I measure how often it's right. An independent evaluator agent audits samples of the finished book, and the reviewer makes the judgement calls. The book states its measured numbers instead of promising perfection.

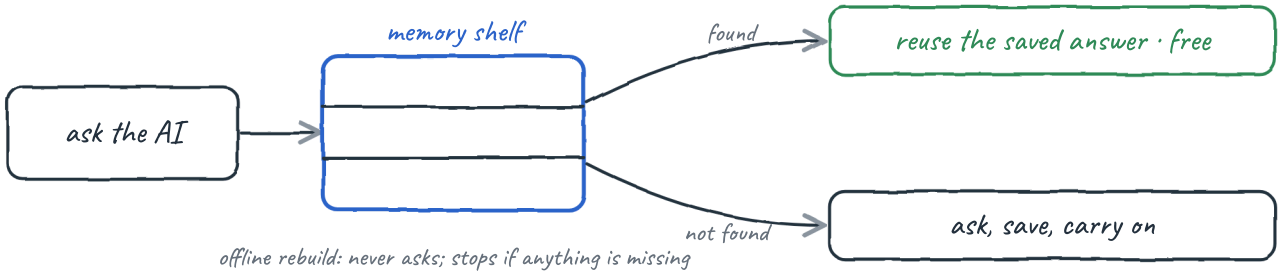
14 The evaluator agent



The checker inside the pipeline looks at every sentence as it is written. The evaluator agent comes afterwards, uses a different AI, never sees what the checker decided, and judges the finished book on the numbers that matter for release. Where both agree, confidence is high; where they disagree, the reviewer looks.

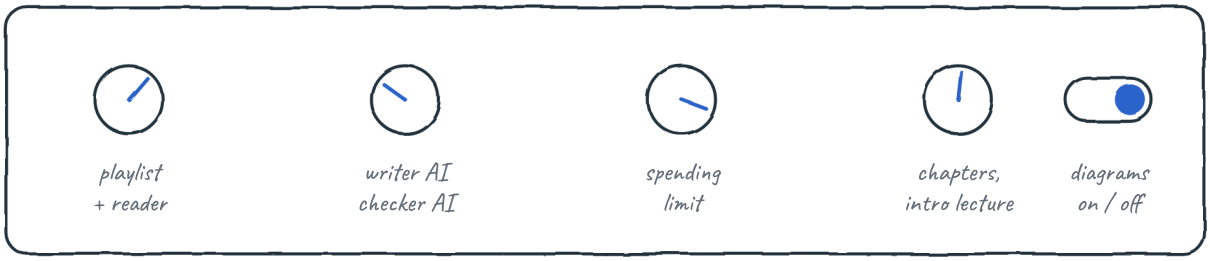
- ✓ facts backed by their lines: **at least 95%** (200 sentences, each judged 3 times, majority counts)
- ✓ misleading errors: **at most 2 in 100**
- ✓ core ideas from random clips missing from the book: **at most 10%**
- ✓ reading score: **at least 3.5 out of 5**

15 Rebuild without paying twice



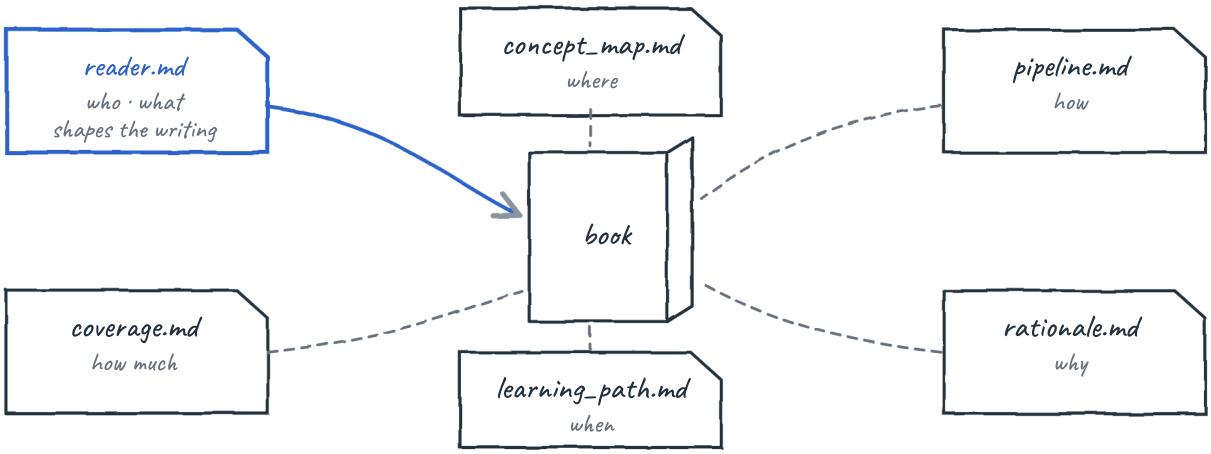
AI answers vary from run to run, so the engine keeps every answer it gets. Anyone with the saved shelf rebuilds exactly the same book in under a second, without paying for a single new answer. Change one thing, such as an instruction or a lecture, and only the parts that depend on it are redone. To share the book without sharing the captions, everything except the raw captions is exported as a book-data package with a fingerprint for every file; the build service rebuilds the book from it on every change and checks that nothing differs.

16 Any course, by settings



Nothing about one particular course is built in. A new playlist needs a new settings card, not new code, and optional parts (like diagrams) switch off cleanly when a course has nothing for them.

17 The 6W brief that travels with every book



- ✓ **reader.md · who / what:** the goal, the reader, what they already know, what is unknown, how the course teaches, and the writing rules. Every inferred line cites evidence from the lectures; an explicit brief overrides it. It is the one page that feeds back into the writing.
- ✓ **coverage.md · how much:** per lecture, how many ideas were found, placed, left out (with reasons) or auto-placed, plus lectures not covered.
- ✓ **concept_map.md · where:** which lectures feed which chapters and sections, with a map.
- ✓ **learning_path.md · when:** the reading order with the source minutes behind each section, and which readiness gates are passed.
- ✓ **pipeline.md · how:** every step, the AI models used, calls, cache reuse and cost.
- ✓ **rationale.md · why:** measured rates against targets, the evaluator agent's verdict, and what still blocks release.

18 Things to try with the finished book



Hover any paragraph

In the web edition, each paragraph's marker opens its sources: the lecture, the minute, a short quote, and a link that plays that exact moment.



Run the code

The listings run for real. Their output is printed in the book, and numbers the text states (a vocabulary size, a parameter count) are checked against what the code computes.



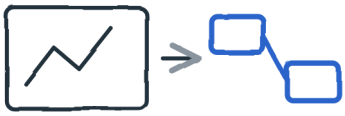
Count the caught lies

The report shows how many deliberately planted false sentences the checker caught, by type, and lists the ones it missed.



Ask "why?"

Point at any sentence and see its whole story: the lecture moment, the ideas behind it, both checks, any rewrite and the reviewer's decision.



See the board, redrawn

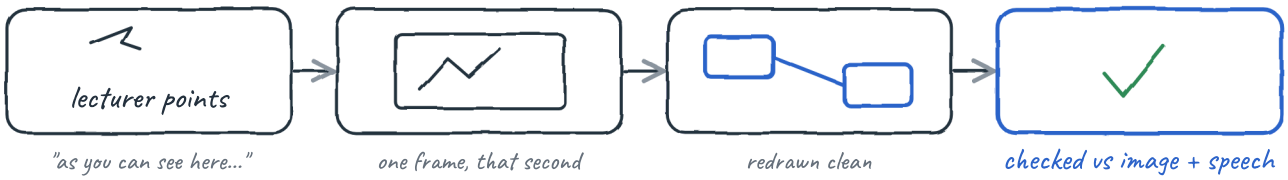
Sketches the lecturer drew on screen come back as clean figures, each captioned with the second of the video it was taken from.



...and the coverage map

A grid of lectures against chapters shows where every part of the course ended up in the book.

18½ What happens to diagrams explained on screen



16 moments looked at · 6 figures kept · each captioned with its second of video

Captions only carry what was said. When a lecture explains a diagram or an equation on the screen, the engine looks for the moment the lecturer points at it, grabs one still frame at that second, has an AI with vision redraw it as a clean diagram or equation, and then checks the redraw twice: against the frame itself and against what is said at that moment. Anything the board doesn't show is dropped. In the first book, 16 moments were read and 6 figures kept; one redraw was rejected for adding a detail that wasn't on the board.

- ✗ Only a few sampled moments per chapter are read, so most on-screen visuals reach the book only through what the lecturer says.
- ✗ Code shown on screen is not read from the video; where the course has a companion repository, listings come from it word for word and are run.
- ✗ It is opt-in, because it downloads video. The second course was built without it.

19 Limits, and what is now handled

- ✓ **Code shown only on screen → listings from the course's reference code.** Sections that teach code get listings taken word-for-word from a pinned reference repository, matched by search, picked by the writer, confirmed by the checker, and labelled with file, licence and version. No code is written by an AI.
- ✓ **Faithful isn't the same as correct → cross-check against the reference.** Technical sentences are compared with the reference. Where they genuinely disagree on a fact or behaviour (not naming or style), the reviewer decides: keep the lecture's version, print a note showing both, or edit. Open disagreements block release.
- ✓ **The checker is an AI → an independent evaluator agent.** A third AI, blind to the checker's verdicts, re-checks a random sample, hunts for missed ideas and scores readability; the reviewer signs off.
- ✓ **Sources that can't be shared → a book-data package.** Everything needed to rebuild the book, minus the raw captions, can be committed. Anyone can verify every file's fingerprint and rebuild the identical book sources; the build service does this on every change and produces the PDF.
- ✓ **Equations and whiteboards shown only on screen → redrawn from the video.** At moments the lecturer points to the screen, a single frame is pulled from the video (opt-in), read by an AI, redrawn as a clean diagram or equation, and checked by a second AI against the image itself and against what the lecturer says. In the first run, 6 board figures were kept and one invented detail was caught and dropped.

★ The first test, two lectures

Before the full run, two lectures went through every step to prove the design on something small:

213 ideas found	**204 · 9** placed · left out with a reason	**2 · 13** chapters · sections	**7,500** words, 6 diagrams
89 → 98% facts backed up, before → after one rewrite	**35 / 40** planted false facts caught	**7** sentences the reviewer had to look at	**~$3** to write the two lectures
0.2 s to rebuild it, free			